

BOOKLEAF

Author Support & Communication Portal

FINAL PRODUCT REQUIREMENTS DOCUMENT

ARCHITECTURE • API • DATABASE • IMPLEMENTATION PHASES • TESTING • DEPLOYMENT •
ANTIGRAVITY BUILD SPECIFICATION

Final Engineering Stack

Frontend: React + TypeScript + Vite
Backend: Python 3.12+ + FastAPI
Validation: Pydantic v2
Database: PostgreSQL + SQLAlchemy 2.x + Alembic
Background jobs: Redis + Celery
Realtime: FastAPI WebSockets
AI: Google Gemini behind an AIProvider abstraction
API: REST + OpenAPI
Testing: Unit + Integration + E2E
Deployment: Docker-compatible

Core differentiators

- Author Communication Timeline — complete chronological operational history.
- Duplicate Ticket Detection & Linking — deterministic, explainable candidate detection with human confirmation.
- ticket_events — the backbone of history, auditability and realtime behavior.

1. Product Definition

The BookLeaf Author Support & Communication Portal is a production-style support operations platform. Authors can view their books, raise support requests and communicate with BookLeaf. Administrators can manage the complete ticket lifecycle, classify and prioritize issues, communicate with authors, add internal notes, inspect author history and resolve duplicate support requests.

The system is a modular monolith, not a microservice collection. Domain modules remain separated internally while deployment remains straightforward.

Primary operational problems

- Royalty and payment questions.
- ISBN and metadata questions.
- Printing and quality issues.
- Distribution and availability issues.
- Book production-status questions.
- General account/support questions.
- Delayed responses, inconsistent communication and growing support backlog.

Product goals

- Give administrators complete context before responding.
- Reduce repeated work caused by duplicate tickets.
- Provide fast, consistent ticket handling.
- Maintain a complete operational history.
- Protect author-level data with resource authorization.
- Keep the UI clean, light and functional.

2. Author Communication Timeline

Every meaningful interaction involving an author is represented chronologically. The timeline is an operational history, not simply a list of messages.

```
08 Oct 10:12 Ticket created
08 Oct 10:12 Classified as ROYALTY_PAYMENT
08 Oct 10:13 Priority set to HIGH
08 Oct 10:15 Assigned to Operations
08 Oct 11:04 Admin response sent
08 Oct 11:10 Duplicate candidate detected: BL-1029
08 Oct 11:12 Ticket linked to BL-1029
09 Oct 14:20 Ticket resolved
```

Ticket event types

- TICKET_CREATED
- MESSAGE_ADDED
- STATUS_CHANGED
- CATEGORY_CHANGED
- PRIORITY_CHANGED
- ASSIGNED
- INTERNAL_NOTE_ADDED
- DUPLICATE_DETECTED
- DUPLICATE_LINKED
- DUPLICATE_UNLINKED
- RESPONSE_SENT
- RESOLVED
- CLOSED

Events are append-only and never edited. Current ticket state remains directly stored for fast querying. This is not full event sourcing; it is current-state persistence plus append-only operational history.

3. Duplicate Ticket Detection

When an author creates a ticket, the backend checks for potentially related existing tickets. Duplicate detection is deterministic and explainable and does not depend on Gemini or any other LLM.

```
New ticket
↓
Author scope
↓
Same book / account scope
↓
Open or recently resolved tickets
↓
Subject normalization
↓
Description normalization
↓
Candidate retrieval
↓
Text similarity
↓
Business rules
↓
Candidate list
```

Signals

- Same author.
- Same book.
- Subject similarity.
- Description similarity.
- Category where available.
- Existing status.
- Recency.

Example

Existing: BL-1012 – Royalty for Code & Karma has not been received
New: BL-1057 – Still waiting for Code & Karma royalty

```
Candidate:
BL-1012
Status: IN_PROGRESS
Similarity: 0.87
Signals: same author + same book + similar issue
```

The new ticket is never silently discarded. The administrator decides whether to link it, continue separately or reject the candidate.

4. Duplicate Relationship

A duplicate is a relationship, not a deletion. Use a relationship table so future relationship types can be supported without changing the ticket model.

```
ticket_relationships
-----
id
source_ticket_id
target_ticket_id
relationship_type
detected_by
confirmed_by
created_at

relationship_type = DUPLICATE
```

The original ticket remains authoritative. The duplicate keeps its own messages and history.

5. Author Portal

Dashboard

- Total books
- Published books
- Books in production
- Total royalty earned
- Royalty paid
- Royalty pending
- Open tickets

My Books

Title
ISBN
Genre
Publication Date
Status
MRP
Copies Sold
Royalty Earned
Royalty Paid
Royalty Pending
Print Partner
Available On

Null and zero values must be handled safely because supplied books can be in production or contain zero/missing financial values.

Create Ticket

Book / General Account Level
Subject
Description
Attachment (optional for assignment)

Ticket creation flow

Persist ticket
↓
Generate ticket number
↓
Create TICKET_CREATED
↓
Queue classification / priority
↓
Run duplicate processing
↓
Return ticket immediately

Author Ticket View

- Ticket number
- Subject
- Book
- Status
- Priority
- Category
- Created date
- Conversation
- Timeline
- Related tickets

Authors see their own messages and admin responses. Internal notes are never returned from author APIs.

6. Admin Portal

Admin Queue

- Status
- Category
- Priority
- Date
- Assignee
- Author
- Search
- Pagination
- Oldest unresolved
- Highest priority
- Recently updated

Admin Ticket Workspace

Author
Book
Ticket details
Current state
Classification
Priority
Conversation
Author timeline
Possible duplicates
Related tickets
Internal notes
Assignment
Response draft

The workspace minimizes context switching. The operator should be able to understand and act on a ticket from one operational surface.

7. Classification, Priority & Response Drafting

Categories

ROYALTY_PAYMENT
ISBN_METADATA
PRINTING_QUALITY
DISTRIBUTION_AVAILABILITY
BOOK_STATUS
GENERAL

Priority

CRITICAL
HIGH
MEDIUM
LOW

Store both system and final values, plus confidence and source. The administrator can override system classification or priority.

Response drafting

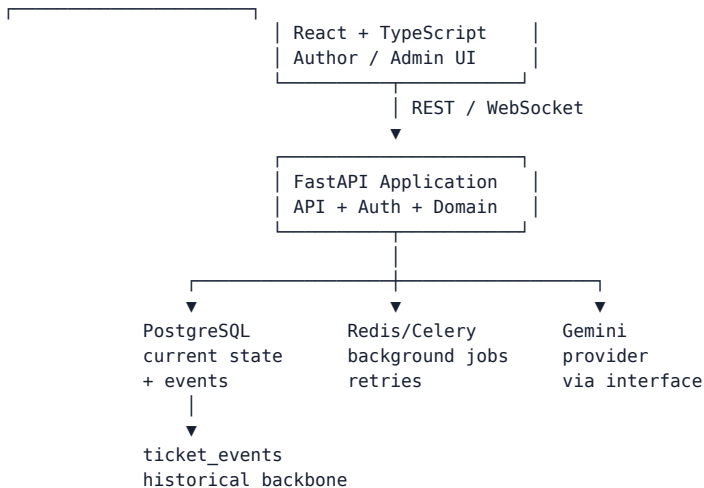
Generate a draft using the ticket, author, relevant book, recent conversation and relevant BookLeaf knowledge-base rules. The administrator reviews, edits and sends the final message. A generated draft is never automatically sent.

Response safety

- Never invent payment dates.
- Never invent refunds.
- Never invent operational or production status.
- Never invent distribution status.
- Never make commitments outside BookLeaf policy.

- If information is unavailable, instruct the administrator to verify manually.

8. Final Architecture



Why PostgreSQL

The domain contains strong relationships between users, authors, books, tickets, messages, notes, assignments, events and relationships. PostgreSQL gives relational integrity, transactions, constraints, indexes and useful text-search capabilities for deterministic duplicate detection.

Why Redis + Celery

Use Redis only for background jobs, retries, rate limiting and limited transient caching. It is never the primary database. Slow AI/provider operations must not block ticket creation.

Why FastAPI

FastAPI provides a clean Python API layer, strong Pydantic validation, native OpenAPI generation and an appropriate foundation for asynchronous HTTP/WebSocket workloads.

Why modular monolith

The assignment does not justify microservice operational complexity. Keep clear domain boundaries inside one backend. Future extraction can happen if scale or organizational boundaries later justify it.

9. Backend Module Structure

```
backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── dependencies.py
│   │   ├── router.py
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── author.py
│   │       ├── books.py
│   │       ├── tickets.py
│   │       ├── admin.py
│   │       └── timeline.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── logging.py
│   │   ├── errors.py
│   │   └── request_context.py
│   ├── db/
│   │   ├── session.py
│   │   ├── base.py
│   │   └── models/
│   └── modules/
│       ├── auth/
│       ├── authors/
│       └── books/
```

```

|   |   | tickets/
|   |   | timeline/
|   |   | relationships/
|   |   | support_assist/
|   |   | knowledge_base/
|   |   | notifications/
|   |   | admin/
|   |   └─ workers/
|   |       │ celery_app.py
|   |       └─ tasks/
|   └─ tests/
|   └─ scripts/
|   └─ alembic.ini
|   └─ pyproject.toml
|   └─ docker-compose.yml

```

Layering

```

FastAPI route
  ↓
Application/service layer
  ↓
Repository/data access
  ↓
SQLAlchemy
  ↓
PostgreSQL

```

Controllers/routes remain thin. Business rules live in services. Database access is isolated. Pydantic schemas define API contracts; SQLAlchemy models are not exposed directly.

10. Frontend Structure

```

frontend/
├─ src/
│   ├─ app/
│   ├─ components/
│   ├─ features/
│   │   ├─ auth/
│   │   ├─ books/
│   │   ├─ tickets/
│   │   └─ timeline/
│   └─ admin/
├─ hooks/
├─ lib/
├─ routes/
├─ types/
├─ package.json
└─ vite.config.ts

```

11. Database Model

users
authors
books
tickets
ticket_messages
ticket_internal_notes
ticket_events
ticket_relationships
ticket_assignments
support_assist_runs
audit_logs

tickets

id
ticket_number
author_id
book_id
subject
description
status
category
priority
system_category
system_category_confidence
category_source
system_priority
system_priority_confidence
priority_source
assigned_admin_id
resolved_at
closed_at
created_at
updated_at

ticket_events

id
ticket_id
actor_user_id nullable
event_type
metadata JSONB
created_at

ticket_relationships

id
source_ticket_id
target_ticket_id
relationship_type
detected_by
confirmed_by
created_at

Integrity

- Foreign keys.
- Unique ticket numbers.
- Unique source/target/type relationship.
- Indexes on author, book, status, priority, category, assignee and timestamps.
- Index ticket_events by ticket_id and created_at.
- Timezone-aware timestamps.
- JSONB event metadata.
- No normal hard deletion of tickets.

12. Ticket State Machine

OPEN
↓
IN_PROGRESS

↓
RESOLVED
↓
CLOSED

State transitions are centralized. Invalid transitions are rejected. State changes and their corresponding event are committed in one transaction.

13. Ticket Events as the Backbone

User action
↓
FastAPI endpoint
↓
Authorization
↓
Domain validation
↓
BEGIN TRANSACTION
├─ update current state
└─ append ticket_event
↓
COMMIT
↓
publish realtime event
↓
WebSocket clients

This design gives fast current-state reads while preserving a complete operational history. Realtime delivery is downstream of successful persistence.

14. Event Metadata Examples

```
STATUS_CHANGED
{
  "old_status": "OPEN",
  "new_status": "IN_PROGRESS"
}

PRIORITY_CHANGED
{
  "old_priority": "MEDIUM",
  "new_priority": "HIGH",
  "source": "ADMIN"
}

DUPLICATE_DETECTED
{
  "candidate_ticket_id": "...",
  "score": 0.87,
  "signals": {
    "same_author": true,
    "same_book": true,
    "subject_similarity": 0.81,
    "description_similarity": 0.69
  }
}
```

15. REST API Contract

Base path: /api/v1

Authentication

POST /auth/login
GET /auth/me
POST /auth/logout

Author

GET /author/profile
GET /author/books
GET /author/books/{book_id}
GET /author/tickets
POST /author/tickets
GET /author/tickets/{ticket_id}
POST /author/tickets/{ticket_id}/messages
GET /author/timeline

Admin

GET /admin/tickets
GET /admin/tickets/{ticket_id}
GET /admin/tickets/{ticket_id}/timeline
GET /admin/tickets/{ticket_id}/relationships
PATCH /admin/tickets/{ticket_id}/status
PATCH /admin/tickets/{ticket_id}/category
PATCH /admin/tickets/{ticket_id}/priority
PATCH /admin/tickets/{ticket_id}/assignment
POST /admin/tickets/{ticket_id}/messages
POST /admin/tickets/{ticket_id}/internal-notes
POST /admin/tickets/{ticket_id}/draft-response
POST /admin/tickets/{ticket_id}/link-duplicate
POST /admin/tickets/{ticket_id}/unlink-duplicate
POST /admin/tickets/{ticket_id}/confirm-not-duplicate
GET /admin/authors/{author_id}/timeline

Ticket creation example

POST /api/v1/author/tickets
Authorization: Bearer <token>
Content-Type: application/json

```
{
  "bookId": "BK002",
  "subject": "My royalty payment is pending",
  "description": "I can see royalty pending for The Saffron Diaries and would like to know when it will be paid."
}
```

Response:

```
{
  "data": {
    "id": "ticket_01J...",
    "ticketNumber": "BL-10024",
    "status": "OPEN",
    "category": null,
    "priority": null,
    "createdAt": "2026-10-08T10:15:00Z"
  }
}
```

Duplicate candidate response

GET /api/v1/admin/tickets/BL-10024/relationships

```
{
  "data": {
    "candidates": [
      {
        "ticketNumber": "BL-10018",
        "subject": "Waiting for royalty payment",
        "status": "OPEN",
        "similarity": 0.87,
        "reason": "Same author, same book, similar issue"
      }
    ]
  },
  "links": {
    "self": "/api/v1/admin/tickets/BL-10024/relationships",
    "parent": "/api/v1/admin/tickets/BL-10024"
  }
}
```

```
    "confirmedRelationships": []  
  }  
}
```

Error contract

```
{  
  "success": false,  
  "error": {  
    "code": "BOOK_NOT_ACCESSIBLE",  
    "message": "The requested book is not available for this account."  
  },  
  "requestId": "req_01J..."  
}
```

Use appropriate 401, 403, 404, 409, 422, 429 and 500 responses.

16. Security & Authorization

- Secure password hashing.
- JWT/access-token authentication.
- AUTHOR and ADMIN roles.
- Resource-level authorization.
- Strict request validation.
- Rate limiting.
- Restricted CORS.
- Secure headers.
- Server-side secret management.
- Request IDs.
- Structured logging.
- No production stack traces.

Author isolation

Authenticated user

↓

Author identity

↓

Permitted resources

Never:

request.authorId → direct ownership decision

An author must not be able to access another author's books, tickets, messages or timeline even if the other resource ID is known.

Internal notes

Internal notes use a separate table and are excluded at the backend DTO/query layer from author endpoints. Frontend hiding is not considered security.

17. Realtime Architecture

Persist transaction

↓

Create ticket_event

↓

COMMIT

↓

Publish WebSocket event

↓

React updates visible state

WebSocket disconnect

↓

Reconnect

↓

Fetch authoritative REST state

Realtime events

ticket.message.created
ticket.status.updated
ticket.assignment.updated
ticket.priority.updated
ticket.category.updated
ticket.relationship.created
timeline.event.created

18. AI Provider Architecture

SupportAssistService

↓

AIProvider interface

↓
GeminiProvider
↓
Gemini API

The application must not import or call the Gemini SDK from ticket controllers or unrelated modules. Provider-specific behavior is isolated inside the provider implementation.

GEMINI_API_KEY=
GEMINI_MODEL=

If Gemini is unavailable, the ticket remains valid. Classification/priority can remain null and the administrator can proceed manually.

19. Complete Step-by-Step Implementation Plan

Phase 0 — Repository Audit & Foundation

- Inspect the entire repository before modifying it.
- Identify existing frontend, backend, database, authentication, deployment and environment configuration.
- Reuse correct existing implementation; do not create parallel implementations.
- Establish final frontend/backend structure and development commands.
- Create .env.example without secrets.
- Create health and readiness endpoints.
- Configure structured logging and request IDs.
- Acceptance: application boots and architecture is documented.

Phase 1 — Database & Seed

- Create SQLAlchemy 2.x models.
- Create Alembic migrations.
- Add foreign keys, unique constraints and indexes.
- Implement unique human-readable ticket numbers.
- Implement idempotent seed from the supplied JSON.
- Seed 10 authors and 18 books plus development users.
- Preserve null and zero values.
- Validate seed counts and ownership relationships.
- Acceptance: migration + repeated seed produces a clean deterministic database.

Phase 2 — Authentication & Authorization

- Implement secure email/password login.
- Hash passwords securely.
- Create authenticated user context.
- Implement AUTHOR and ADMIN roles.
- Implement resource-level authorization.
- Test author A against author B resources.
- Never trust client authorId for ownership.
- Acceptance: authentication and isolation are enforced server-side.

Phase 3 — Author Books

- Implement profile, books and book detail APIs.
- Build login, dashboard, books and book detail screens.
- All data comes from APIs.
- Handle null/zero financial and production fields.
- Acceptance: each author sees only their own books.

Phase 4 — Core Ticket Domain

- Implement POST /author/tickets.
- Validate book ownership.
- Generate unique ticket number.
- Persist ticket.
- Create TICKET_CREATED event in the same transaction.
- Implement author ticket list/detail.
- Acceptance: ticket creation is real and durable.

Phase 5 — Conversation

- Implement author/admin messages.

- Persist messages separately.
- Create MESSAGE_ADDED events.
- Implement internal notes separately.
- Ensure author endpoints never return internal notes.
- Acceptance: real two-way communication works.

Phase 6 — Ticket Event Engine

- Implement TicketEventService.
- Standardize event types and metadata.
- Make events append-only.
- Use the service from all important business operations.
- Implement author and admin timeline queries.
- Add pagination.
- Acceptance: history is complete and consistent.

Phase 7 — Ticket State Machine

- Implement OPEN → IN_PROGRESS → RESOLVED → CLOSED.
- Reject invalid transitions.
- Record old/new values.
- Commit state + event transactionally.
- Acceptance: lifecycle cannot be corrupted.

Phase 8 — Admin Queue

- Implement admin ticket list.
- Add status/category/priority/assignee/author/date filters.
- Add search and pagination.
- Add sorting by age, priority and recent update.
- Build clean table-first queue.
- Acceptance: admin can efficiently locate work.

Phase 9 — Gemini Classification & Priority

- Create AIProvider interface.
- Implement GeminiProvider.
- Keep key server-side.
- Create Celery tasks for classification and priority.
- Validate structured provider responses.
- Store system value, confidence and source.
- Allow admin override.
- Acceptance: provider failure never breaks ticket creation.

Phase 10 — Response Drafting

- Create maintainable BookLeaf knowledge base.
- Select relevant policy context.
- Send ticket/book/conversation context to Gemini.
- Validate generated draft.
- Show editable response.
- Send only admin-approved final text.
- Acceptance: AI is assistive, never authoritative.

Phase 11 — Duplicate Detection

- Scope candidate tickets by author and book/account.
- Retrieve open/recent candidates.

- Normalize subject and description.
- Calculate text similarity.
- Apply recency/status/business rules.
- Store explainable signals.
- Do not use an LLM.
- Acceptance: candidates are deterministic and reviewable.

Phase 12 — Duplicate Relationships

- Implement ticket_relationships.
- Add link/unlink/confirm-not-duplicate endpoints.
- Use uniqueness constraints.
- Keep both histories.
- Create relationship timeline events.
- Acceptance: human-confirmed relationships are safe and reversible.

Phase 13 — Redis & Celery

- Configure Redis.
- Configure Celery worker.
- Create background tasks.
- Implement retry policy.
- Make tasks idempotent.
- Log task state and failure.
- Acceptance: slow processing is asynchronous and recoverable.

Phase 14 — WebSockets

- Implement authenticated WebSocket connections.
- Publish only after database commit.
- Broadcast ticket/message/status/relationship/timeline changes.
- Implement reconnect.
- Refresh REST state after reconnect.
- Acceptance: realtime is reliable but not the source of truth.

Phase 15 — Admin Workspace

- Build a single operational ticket screen.
- Show author, book, state, conversation, timeline, duplicates, internal notes, assignment and response.
- Connect every control to real API mutations.
- Implement loading/empty/error states.
- Acceptance: operator can understand and act from one page.

Phase 16 — Author Workspace

- Build dashboard, books, tickets, ticket detail and timeline.
- Make timeline the central support-history surface.
- Show relationship information safely.
- Never expose internal notes.
- Acceptance: author can understand support history without manually opening every ticket.

Phase 17 — Security Hardening

- Review authentication, authorization, CORS, headers, rate limits and secrets.
- Perform IDOR tests.
- Test internal-note privacy.
- Test malformed requests.
- Ensure production errors do not leak internals.

- Acceptance: deliberate negative tests fail safely.

Phase 18 — Testing

- Unit test domain services.
- Integration test API workflows.
- Test ticket event creation.
- Test duplicate scoring/linking.
- Test Gemini failure.
- Test seed validation.
- Add E2E critical path.
- Acceptance: core workflows pass end-to-end.

Phase 19 — Documentation

- Write README.
- Document architecture.
- Document API/OpenAPI.
- Document environment variables.
- Document migrations and seed.
- Document realtime.
- Document duplicate detection.
- Document Gemini provider abstraction.
- Document trade-offs and limitations.
- Acceptance: another engineer can clone and run the system.

Phase 20 — Deployment & Production Readiness

- Create Docker configuration.
- Configure frontend/API/database/worker environments.
- Configure migrations and health checks.
- Implement graceful shutdown.
- Run lint, type checks and tests.
- Verify deployed frontend/API/WebSocket behavior.
- Acceptance: deployment is repeatable and production-safe.

20. Testing Matrix

Unit tests

- Authorization service.
- Ticket state machine.
- TicketEventService.
- Duplicate scoring.
- Relationship validation.
- Provider response validation.
- Knowledge-base selection.

Integration tests

- Login.
- Author book access.
- Ticket creation.
- Admin access.
- Messages.
- Internal notes.
- Status/category/priority updates.
- Timeline.
- Duplicate link/unlink.
- Gemini failure.

End-to-end test

Author login
→ view books
→ create ticket
→ TICKET_CREATED
→ background classification/priority
→ duplicate candidate
→ admin opens ticket
→ admin reviews timeline
→ admin responds
→ author receives realtime response
→ timeline updates
→ ticket resolved

Mandatory negative tests

- Author A cannot read Author B's ticket.
- Author A cannot read Author B's book.
- Author cannot read internal notes.
- Invalid ticket transition is rejected.
- Duplicate relationship cannot be inserted twice.
- Gemini outage does not prevent ticket creation.
- Malformed Gemini output is rejected.
- WebSocket failure does not corrupt database state.

21. UI Specification

Visual direction

- Light theme.
- White surfaces.
- Subtle borders.
- Neutral typography.
- Compact spacing.

- Clear status/priority badges.
- Table-first admin queue.
- Timeline-first author history.
- Responsive layout.

Do not add

- AI badges.
- Robot graphics.
- Glowing gradients.
- Fake statistics.
- Marketing hero sections.
- Chatbot UI.
- Unnecessary animation.
- Decorative explanatory text.
- Hardcoded ticket/book/message records.

Every data screen must support

- Loading state.
- Empty state.
- Error state.
- Successful mutation feedback.
- Pagination where required.
- Realtime update handling.

22. Deployment & Operations

```
React frontend
  ↓
Frontend hosting
  ↓
FastAPI API
  ↓
PostgreSQL
Redis
Celery worker
```

Environment

```
DATABASE_URL=
REDIS_URL=
JWT_SECRET=
GEMINI_API_KEY=
GEMINI_MODEL=
CORS_ORIGINS=
APP_ENV=
LOG_LEVEL=
```

Production behavior

- No secrets in repository.
- No secrets in frontend bundle.
- Migrations are versioned.
- Health/readiness endpoints exist.
- Workers shut down gracefully.
- Database connections close cleanly.
- Request IDs appear in logs.
- Provider failures are observable.
- Production errors do not expose stack traces.

23. Seed & Developer Workflow

```
# backend
python -m venv .venv
pip install -r requirements.txt

alembic upgrade head
python scripts/seed.py

# run API
uvicorn app.main:app --reload

# worker
celery -A app.workers.celery_app worker --loglevel=info

# frontend
npm install
npm run dev
```

Use the project's actual selected package manager and scripts if the repository already has an established convention. Do not duplicate command systems.

24. Documentation Deliverables

- README.md — setup, environment, seed, tests, deployment.
- docs/architecture.md — architecture and data flow.
- docs/api.md — endpoint behavior and examples.
- docs/decisions.md — major engineering trade-offs.

25. Final Antigravity Master Build Prompt

Paste the following after this PRD. It is the implementation instruction set for Antigravity.

Use this PRD as the product and functional source of truth.

Build the BookLeaf Author Support & Communication Portal as a real production-style application.

FINAL STACK:

Frontend: React + TypeScript + Vite

Backend: Python 3.12+ + FastAPI

Validation: Pydantic v2

ORM: SQLAlchemy 2.x

Migrations: Alembic

Database: PostgreSQL

Queue: Redis + Celery

Realtime: FastAPI WebSockets

AI: Google Gemini behind an AIProvider abstraction

API: REST + OpenAPI

Testing: unit + integration + E2E

Deployment: Docker-compatible

Do not require an OpenAI API key.

Use Gemini initially.

Keep provider-specific code isolated.

FIRST:

Audit the existing repository before modifying anything.

Identify existing frontend, backend, database, authentication, deployment and configuration.

Reuse correct existing code.

Do not create parallel implementations.

MOST IMPORTANT ARCHITECTURAL RULE:

Make ticket_events the backbone of the product.

This is NOT full event sourcing.

Keep current ticket state in tickets.

Keep append-only operational history in ticket_events.

Every meaningful business operation must create a ticket event.

EVENT TYPES:

TICKET_CREATED

MESSAGE_ADDED

STATUS_CHANGED

CATEGORY_CHANGED

PRIORITY_CHANGED

ASSIGNED

INTERNAL_NOTE_ADDED

DUPLICATE_DETECTED

DUPLICATE_LINKED

DUPLICATE_UNLINKED

RESPONSE_SENT

RESOLVED

CLOSED

Create one TicketEventService.

Controllers must not write arbitrary event rows.

For important mutations:

validate

BEGIN TRANSACTION

update current state

append event

audit if required

COMMIT

then publish realtime.

DUPLICATE DETECTION:

Do NOT use Gemini.

Use deterministic signals:

same author

same book

subject similarity

description similarity

category

status

recency

Produce explainable candidate scores.
Never silently delete or merge tickets.
Admin must confirm relationships.

RELATIONSHIP:
ticket_relationships
source_ticket_id
target_ticket_id
relationship_type
detected_by
confirmed_by
created_at

AI:
Create AIProvider and GeminiProvider.
Use Gemini for:
classification
priority
response drafting

Use:
GEMINI_API_KEY
GEMINI_MODEL

Never expose credentials to React.
Ticket creation must work without Gemini.
Use Celery/Redis for slow processing.
Validate provider output.
Generated responses require admin approval.

BACKEND:
Use modular FastAPI structure:
api
core
db
modules
workers

Modules:
auth
authors
books
tickets
timeline
relationships
support_assist
knowledge_base
notifications
admin

Keep routes thin.
Business rules in services.
Persistence in repositories.
Pydantic schemas for API contracts.

DATABASE:
users
authors
books
tickets
ticket_messages
ticket_internal_notes
ticket_events
ticket_relationships
ticket_assignments
support_assist_runs
audit_logs

AUTHORIZATION:
AUTHOR and ADMIN.
Use resource-level authorization.
Never trust client authorId.
Test IDOR.

API BASE:
/api/v1

AUTHOR:
GET /author/profile

GET /author/books
GET /author/books/{book_id}
GET /author/tickets
POST /author/tickets
GET /author/tickets/{ticket_id}
POST /author/tickets/{ticket_id}/messages
GET /author/timeline

ADMIN:

GET /admin/tickets
GET /admin/tickets/{ticket_id}
GET /admin/tickets/{ticket_id}/timeline
GET /admin/tickets/{ticket_id}/relationships
PATCH /admin/tickets/{ticket_id}/status
PATCH /admin/tickets/{ticket_id}/category
PATCH /admin/tickets/{ticket_id}/priority
PATCH /admin/tickets/{ticket_id}/assignment
POST /admin/tickets/{ticket_id}/messages
POST /admin/tickets/{ticket_id}/internal-notes
POST /admin/tickets/{ticket_id}/draft-response
POST /admin/tickets/{ticket_id}/link-duplicate
POST /admin/tickets/{ticket_id}/unlink-duplicate
POST /admin/tickets/{ticket_id}/confirm-not-duplicate
GET /admin/authors/{author_id}/timeline

FRONTEND:

Build login, author dashboard, books, book detail, tickets, ticket detail, timeline, create ticket, admin queue, admin workspace and a

Every displayed record must come from real APIs.
No hardcoded production records.
No fake endpoints.

UI:

light enterprise style
white surfaces
subtle borders
compact spacing
clear badges
no AI branding
no robot graphics
no glowing gradients
no fake dashboards
no chatbot UI
no unnecessary animation

REALTIME:

Database is source of truth.
Persist first, then publish.
On reconnect, refetch REST state.

SEED:

Use supplied BookLeaf JSON.
Expected: 10 authors, 18 books.
Preserve null and zero values.
Seed must be idempotent.

TEST:

authentication
authorization
author isolation
ticket lifecycle
ticket events
timeline
duplicate detection
duplicate linking
internal note privacy
Gemini failure
API validation
seed
E2E critical path

FAILURE:

Gemini failure cannot block ticket creation.
Redis/Celery failure cannot corrupt persisted state.
WebSocket failure cannot corrupt persisted state.
Duplicate processing failure cannot invalidate ticket creation.

CODE QUALITY:

strict typing

small modules
clear names
no giant files
no giant functions
no duplicated business rules
no raw provider calls throughout the application
explicit error handling
transactions
tests

Do not create files/comments mentioning:

ChatGPT
AI generated
Gemini generated
generated by AI
assistant output

Do not put AI marketing text into the UI.

IMPLEMENT IN ORDER:

- 0 repository audit
- 1 FastAPI foundation
- 2 PostgreSQL/SQLAlchemy/Alembic
- 3 schema
- 4 seed
- 5 authentication
- 6 authorization
- 7 books
- 8 tickets
- 9 messages
- 10 ticket event engine
- 11 state machine
- 12 timeline
- 13 admin queue
- 14 duplicate detection
- 15 duplicate relationships
- 16 Redis/Celery
- 17 Gemini provider
- 18 classification
- 19 priority
- 20 response drafting
- 21 WebSockets
- 22 author frontend
- 23 admin frontend
- 24 realtime UI
- 25 security
- 26 tests
- 27 documentation
- 28 Docker/deployment

After each phase:

format
lint
typecheck
test
verify imports
verify migrations
fix errors
then continue.

FINAL QUALITY:

No mock endpoints.
No fake API responses.
No hardcoded production data.
No placeholder critical features.
No missing UI for implemented APIs.
No UI action that does nothing.
No unhandled critical failure.
No claim of functionality that is not implemented.

The final result must be a real, testable, deployable BookLeaf support operations platform.

26. Final Acceptance Checklist

- Author can log in securely.
- Author sees only their own books.
- Author can create a ticket.
- Ticket gets a unique number.
- TICKET_CREATED is stored transactionally.
- Messages are persisted.
- Internal notes are private.
- Ticket lifecycle is validated.
- Admin can filter and manage tickets.
- Classification works through Gemini provider abstraction.
- Priority works through Gemini provider abstraction.
- Admin can override classification and priority.
- Response drafting uses relevant BookLeaf knowledge.
- Generated response requires admin approval.
- Duplicate candidates are deterministic and explainable.
- Duplicate relationships are human-confirmed.
- Duplicate relationships are unique and reversible.
- Both duplicate tickets retain their own histories.
- Author timeline is backed by ticket_events.
- Admin author history is available.
- Realtime updates work.
- Reconnect restores authoritative REST state.
- Gemini failure does not break ticket creation.
- Background jobs are retry-safe.
- Author-to-author data isolation is tested.
- Seed creates 10 authors and 18 books idempotently.
- OpenAPI documentation is available.
- Tests pass.
- README and architecture documentation are accurate.
- Docker/local setup works.
- Production secrets are not exposed.
- UI is clean, light and operational.

27. Engineering Positioning

The project should feel like an operational support system with intelligence, not a CRUD application with an LLM attached. The strongest architectural story is the combination of transactional current state, append-only ticket events, deterministic duplicate detection, strict author isolation, asynchronous AI assistance and realtime delivery.

```
Current ticket state
+
Append-only ticket_events
+
Ticket relationships
+
Provider-isolated AI
+
Realtime delivery
```

= coherent support operations platform

Interview-level architectural statement: “Current ticket state is optimized for operational reads; ticket_events preserve the complete operational history; relationships model repeated issues; AI is an isolated assistive subsystem and never the source of truth.”

END OF FINAL ENGINEERING SPECIFICATION